# Software Requirements Specification

## for

# Campus Food Ordering and Management System

**Version <1.0>**

**Group No.:** 1

| | |
|---|---|
| **< Hassan Abdelhaleem Hassan Basheir >** | **< 251UC2506T >** |
| **< Rashad Ali Qaid Sofan >** | **< 251UC2505G >** |
| **< Ahmad Aljammal >** | **< 251UC2500R >** |
| **< Mohammed A. M. Alakklouk >** | **< 251UC250CR >** |


**Date: 31/04/2026**

# Contents

# Revisions

| Version | Primary Author(s) | Description of Version | Date Completed |
|---|---|---|---|
| SRS in Part 1(as Ver 1.0)<br>SDS in Part 2(as Ver 2.0.X)<br>*System Documentation in Part 3 (as Ver 3.0)<br>Draft Type and Number | Full Name | Information about the revision. This table does not need to be filled in whenever a document is touched, only when the version is being upgraded. | 00/00/00 |

# 1 Project Introduction

The Campus Food Ordering and Management System aims to digitize and streamline the food purchasing, vendor management, and queue control processes within the university campus. By integrating pre-ordering, scheduling, real-time tracking, and payment functionalities, the system addresses inefficiencies in traditional campus dining operations.

## 1.1 Team Members

| Name | Actor/Processes |
|---|---|
| Hassan Abdelhaleem Hassan Basheir | Student |
| Rashad Ali Qaid Sofan | Vendor |
| Ahmad Aljammal | Admin |
| Mohammed A. M. Alakklouk | Staff |

## 1.2 Problem statement

Campus food outlets frequently experience overcrowding and long queues during peak meal hours, leading to student frustration, reduced break time, and inefficient vendor operations. Manual order processing, lack of real-time inventory updates, and the absence of a centralized pre-ordering mechanism result in inaccurate wait times, food waste, and poor customer satisfaction. Vendors lack tools to effectively manage digital queues, track sales trends, or optimize menu offerings based on demand. Administrators face challenges in monitoring campus-wide dining operations, vendor compliance, and system performance. A unified digital platform is required to automate order scheduling, enable secure payments, provide transparent queue and order tracking, and deliver actionable analytics to optimize campus dining services.

## 1.3 Project Schedule

| **Main Section / Week** | **Apr 3–9** | **Apr 10–16** | **Apr 17–23** | **Apr 24–27** | **Apr 28–30** |
|---|---|---|---|---|---|
| Project Introduction | | | | | |
| System Overview | | | | | |
| Functional Requirements | | | | | |
| System Models | | | | | |
| Non-Functional Requirements | | | | | |
| References | | | | | |
| Final Review & Submission | | | | | |

# 2 System Overview

## 2.1 Description

The Campus Food Ordering and Management System is a centralized digital platform that facilitates seamless food ordering, vendor management, and operational analytics for the campus community. The system enables students to browse vendor menus, add items to a digital cart, schedule orders with specific pick-up times, complete secure payments, and track order status and queue position in real-time. Vendors can manage their menus, update inventory availability, process incoming orders, and manage digital queue tickets. Administrators oversee user and vendor accounts, configure system settings, monitor campus-wide dining analytics, and generate compliance and performance reports. The platform integrates notification services, role-based access control, and a rating/review mechanism to ensure a transparent, efficient, and user-friendly campus dining experience.

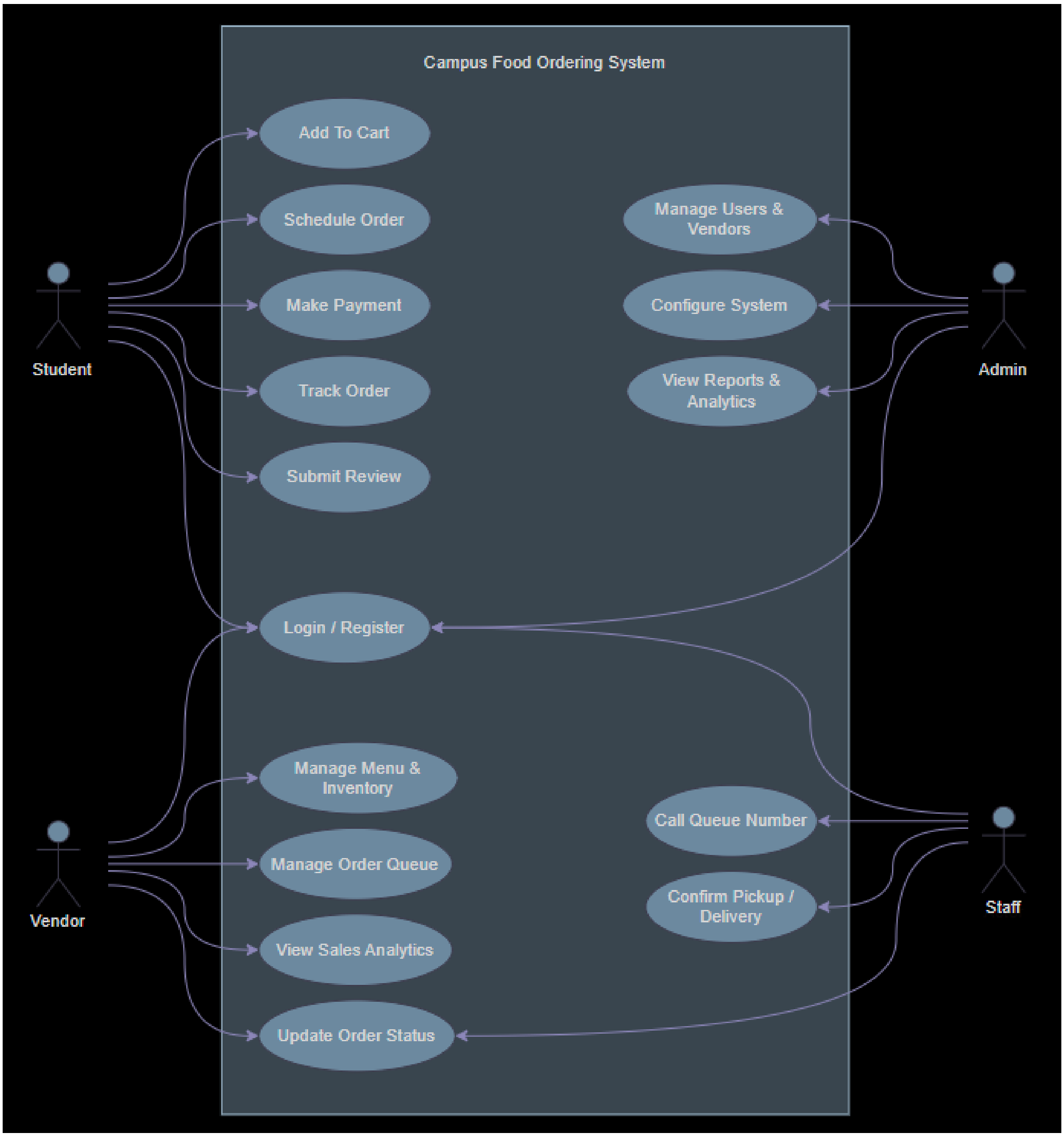
## 2.2 Actors

| Actor | Use Cases |
|---|---|
| **Student** | Register/Login, Browse Menu & Add to Cart, Schedule Order & Select Pick-up Time, Process Payment, Track Order Status & Queue Position, Submit Ratings & Reviews |
| **Vendor** | Register/Login, Manage Food Menu & Inventory, View & Update Order Status, Manage Digital Queue, View Sales & Performance Analytics |
| **Admin** | Manage User & Vendor Accounts, Configure System Settings & Operating Hours, Monitor Campus-wide Analytics & Reports, Handle Support/Disputes, Manage Payment Gateway Integration |
| **Staff** | A staff member responsible for handling order fulfillment operations, including managing the order queue, updating order statuses (e.g., preparing, ready), calling queue numbers, and confirming order pickup. |

## 2.3 Assumptions and Dependencies

- **Assumptions:**
    - All students, vendors, and staff have access to a smartphone or computer with a stable internet connection.
    - Vendors will maintain accurate and up to date menu items, pricing, and availability statuses.
    - The campus network infrastructure supports reliable communication between the application server and external services
    - Peak usage periods will primarily occur during standard campus meal times (e.g., 11:30 AM–2:00 PM, 5:00 PM–7:00 PM).
- **Dependencies:**
    - Third-party payment gateway (e.g. or TnG e-wallet) for secure transaction processing.
    - External notification services.
    - Campus authentication system.
    - Cloud hosting provider for scalable deployment and database backups.

## 2.4 Use Case Diagram



# 3 Functional Requirements

## 3.1 Student

### 3.1.1 Add to Cart

- The system shall allow the user to add a selected food item to the cart.
- The system shall allow the user to choose quantity before adding.
- The system shall display the updated cart after an item is added.
- The system shall calculate and update the total price automatically.
- The system shall prevent adding items that are unavailable.

<table>
<tr><th colspan="2">Use Case Name</th><th>Add to Cart</th></tr>
<tr><td colspan="2">Actors</td><td>Student</td></tr>
<tr><td colspan="2">Preconditions</td><td>Student is logged into the system.</td></tr>
<tr><td rowspan="2">Normal Flow</td><td>Description</td><td>1. The student selects a food item from the menu.<br>2. The student selects the desired quantity.<br>3. The student clicks “Add to Cart.”<br>4. The system checks item availability.<br>5. The system adds the item to the cart.<br>6. The system updates and displays the cart.<br>7. system recalculates the total price.</td></tr>
<tr><td>Postconditions</td><td>1. The selected item is added to the cart.<br>2. The cart is updated with the correct quantity and total price.</td></tr>
<tr><td colspan="2">Alternative flows and exceptions</td><td>• If the item is unavailable, the system displays an error message and does not add the item.<br>• If the selected quantity is invalid (e.g., zero or negative), the system prompts the user to enter a valid quantity.</td></tr>
<tr><td colspan="2">Non functional requirements</td><td>• The cart should update instantly.<br>• The system should retain cart data during the session.</td></tr>
</table>

### 3.1.2 Make Payment

- The system shall allow the student to choose a payment method.
- The system shall calculate and display the total payment amount.
- The system shall process payments securely.
- The system shall confirm successful payment to the student.
- The system shall generate a receipt after payment.

<table>
<tr><th colspan="2">Use Case Name</th><th>Make Payment</th></tr>
<tr><td colspan="2">Actors</td><td>Student</td></tr>
<tr><td colspan="2">Preconditions</td><td>The student is logged into the system and has items in the cart.</td></tr>
<tr><td rowspan="2">Normal Flow</td><td>Description</td><td>1. The student proceeds to the checkout page.<br>2. The system displays the total price.<br>3. The student enters payment details.<br>4. The system processes the payment.<br>5. The system confirms the order.</td></tr>
<tr><td>Postconditions</td><td>The payment is completed and the order is confirmed.</td></tr>
<tr><td colspan="2">Alternative flows and exceptions</td><td>1. If the payment fails, the system displays an error message.<br><br>2. If the payment details are invalid, the system asks the user to re-enter the information.</td></tr>
<tr><td colspan="2">Non functional requirements</td><td>• The system should process payments securely.<br><br>• The system should respond within a few seconds.<br><br>• The interface should be simple and user-friendly.</td></tr>
</table>

### 3.1.3 Track Order

- The system shall allow the student to view the current status of an order.
- The system shall update the order status in real time or periodically.
- The system shall display estimated delivery or pickup time.
- The system shall notify the student when the order status changes.
- The system shall allow tracking only for active orders.

| ***Use Case Name*** | | Track Order |
|---|---|---|
| Actors | | Student |
| Preconditions | | The student is logged into the system and has already placed an order. |
| Normal Flow | Description | 1. The student opens the order tracking page.<br>2. The system displays the current order status.<br>The student views updates and queue position. |
| | Postconditions | The student is informed about the order status. |
| Alternative flows and exceptions | | If no order is found, the system displays a message. |
| Non functional requirements | | • The system should update order status in real-time.<br>• The interface should be simple and user-friendly. |

### 3.1.4 Schedule Order

- The system shall allow the student to choose a future date and time for the order.
- The system shall display available scheduling slots.
- The system shall save the scheduled order details.
- The system shall send a confirmation after scheduling.
- The system shall prevent booking unavailable time slots.

| ***Use Case Name*** | | Schedule Order |
|---|---|---|
| Actors | | Student |
| Preconditions | | The student is logged into the system and has already placed an order. |
| Normal Flow | Description | 1. The student opens the schedule order page.<br>2. The student selects the preferred date and time for delivery/pickup.<br>3. The student confirms the scheduled order details.<br>4. The system saves the scheduled order and displays a confirmation message. |
| | Postconditions | The scheduled order is successfully recorded in the system. |
| Alternative flows and exceptions | | If the selected time slot is unavailable, the system asks the student to choose another time. |
| Non functional requirements | | • The system should save scheduled orders accurately and securely.<br>• The interface should be simple and user-friendly.<br>• The system should respond quickly when displaying available time slots. |

### 3.1.5 Submit Review

- The system shall allow the student to rate completed orders.
- The system shall allow the student to enter written feedback.
- The system shall save submitted reviews in the database.
- The system shall display a confirmation after submission.
- The system shall only allow reviews for completed orders.

| ***Use Case Name*** | | Submit Review |
|---|---|---|
| Actors | | Student |
| Preconditions | | The student is logged into the system and has already completed an order. |
| Normal Flow | Description | 1. The student opens the review page.<br>2. The student selects the completed order to review.<br>3. The student enters a rating and written feedback.<br>4. The student submits the review.<br>5. The system saves the review and displays a confirmation message. |
| | Postconditions | The review is successfully recorded in the system. |
| Alternative flows and exceptions | | If the student leaves required fields empty, the system displays an error message and requests completion. |
| Non functional requirements | | • The system should save reviews securely and accurately.<br>• The interface should be simple and user-friendly.<br>• The system should respond quickly when submitting reviews. |

### 3.1.6 Login / Register

- The system shall allow new users to create an account.
- The system shall allow existing users to log in using valid credentials.
- The system shall validate user input during registration and login.
- The system shall display error messages for invalid details.
- The system shall grant access after successful login or registration.

| ***Use Case Name*** | | Login / Register |
|---|---|---|
| Actors | | Student, Vendor, Admin, Staff |
| Preconditions | | The user has access to the system login page. |
| Normal Flow | Description | 1. The user opens the login/register page.<br>2. The user chooses to log in or create a new account.<br>3. If logging in, the user enters username/email and password.<br>4. If registering, the user enters required personal details and password.<br>5. The system validates the information.<br>6. The system logs in the student or creates a new account successfully. |
| | Postconditions | The user gains access to the system with a valid account. |
| Alternative flows and exceptions | | If login credentials are incorrect or registration details are invalid, the system displays an error message and requests correction. |
| Non functional requirements | | • The system should protect user data securely.<br>• The interface should be simple and user-friendly.<br>• The system should respond quickly during login and registration. |

## 3.2 Vendor

### 3.2.1 Manage Menu & Inventory

- The system shall allow the vendor to add new menu items.
- The system shall allow the vendor to edit or remove menu items.
- The system shall allow the vendor to update stock availability.
- The system shall display current inventory levels.
- The system shall prevent customers from ordering unavailable items.

| ***Use Case Name*** | | Manage Menu & Inventory |
|---|---|---|
| Actors | | Vendor |
| Preconditions | | Vendor account is verified and logged in. |
| Normal Flow | Description | 1. Vendor navigates to "Menu Management".<br>2. System lists existing items with status indicators.<br>3. Vendor adds new items (name, price, category, image, prep time).<br>4. Vendor toggles availability or updates stock limits.<br>5. System validates inputs and saves changes.<br>6. Updated menu reflects instantly for students. |
| | Postconditions | Menu database updated. Students see real-time availability changes. |
| Alternative flows and exceptions | | - Invalid image format: System rejects upload and prompts correct format.<br>- Price conflict: System prevents saving if price is below minimum threshold. |
| Non functional requirements | | Menu updates propagate to student view within 3 seconds. Supports bulk CSV import for initial setup. |

### 3.2.2 Manage Order Queue

- The system shall display incoming customer orders to the vendor.
- The system shall allow the vendor to accept or reject orders.
- The system shall allow the vendor to prioritize or manage the queue.
- The system shall update order positions automatically.
- The system shall notify customers of queue changes.

<table>
<tr><th colspan="2">Use Case Name</th><th>Manage Order Queue</th></tr>
<tr><td colspan="2">Actors</td><td>Vendor</td></tr>
<tr><td colspan="2">Preconditions</td><td>The vendor is logged into the system and orders are available.</td></tr>
<tr><td rowspan="2">Normal Flow</td><td>Description</td><td>1. The vendor opens the order queue page.<br>2. The system displays all pending orders.<br>3. The vendor reviews and manages the queue.<br>4. The vendor accepts, rejects, or rearranges orders.<br>5. The system updates the queue status.</td></tr>
<tr><td>Postconditions</td><td>The order queue is successfully updated.</td></tr>
<tr><td colspan="2">Alternative flows and exceptions</td><td>If no orders are available, the system displays a message.</td></tr>
<tr><td colspan="2">Non functional requirements</td><td>• The system should update the queue in real time.<br>• The interface should be simple and user-friendly.<br>• The system should respond quickly when loading orders.</td></tr>
</table>

### 3.2.3 View Sales Analytics

- The system shall allow the vendor to view sales reports.
- The system shall display daily, weekly, or monthly sales data.
- The system shall show total revenue and number of orders.
- The system shall allow filtering reports by date range.
- The system shall present analytics clearly using charts or summaries.

| ***Use Case Name*** | | View Sales Analytics |
|---|---|---|
| Actors | | Vendor |
| Preconditions | | The vendor is logged into the system and sales data exists. |
| Normal Flow | Description | 1. The vendor opens the sales analytics page.<br>2. The system retrieves sales records.<br>3. The vendor selects a date range or report type.<br>4. The system displays charts, summaries, and statistics.<br>5. The vendor reviews the analytics results. |
| | Postconditions | The vendor successfully views sales analytics. |
| Alternative flows and exceptions | | If no sales data is available, the system displays a message. |
| Non functional requirements | | • The system should display reports accurately.<br>• The interface should be simple and user-friendly.<br>• The system should load analytics efficiently. |

### 3.2.4 Update Order Status

- The system shall allow the vendor to change order status.
- The system shall provide status options such as Preparing, Ready, or Completed.
- The system shall notify customers when status changes.
- The system shall save status updates immediately.
- The system shall display the latest order status to users.

| ***Use Case Name*** | | Update Order Status |
|---|---|---|
| Actors | | Vendor, Staff |
| Preconditions | | The user is logged into the system and active orders exist. |
| Normal Flow | Description | 1. The user opens the order management page.<br>2. The user selects an active order.<br>3. The user chooses a new order status.<br>4. The system saves the updated status.<br>5. The customer is notified of the status change. |
| | Postconditions | The selected order status is successfully updated. |
| Alternative flows and exceptions | | If the selected order is invalid, the system displays an error message. |
| Non functional requirements | | • The system should update status in real time.<br>• The interface should be simple and user-friendly.<br>• The system should process updates quickly. |

## 3.3 Admin

### 3.3.1 Manage Users & Vendors

- The system shall allow the admin to view all user and vendor accounts.
- The system shall allow the admin to add, edit, suspend, or delete accounts.
- The system shall allow the admin to approve or reject vendor registrations.
- The system shall display account status and details.
- The system shall restrict account management functions to admin users only.

<table>
<tr><th colspan="2">Use Case Name</th><th>Manage Users & Vendors</th></tr>
<tr><td colspan="2">Actors</td><td>Admin</td></tr>
<tr><td colspan="2">Preconditions</td><td>Admin is logged into the system.</td></tr>
<tr><td rowspan="2">Normal Flow</td><td>Description</td><td>1. The admin opens the user and vendor management page.<br>2. The system displays all accounts.<br>3. The admin selects an account.<br>4. The admin adds, edits, approves, suspends, or deletes the selected account.<br>5. The system saves the changes and displays confirmation.</td></tr>
<tr><td>Postconditions</td><td>User or vendor account information is successfully updated.</td></tr>
<tr><td colspan="2">Alternative flows and exceptions</td><td>If invalid changes are made, the system displays an error message.</td></tr>
<tr><td colspan="2">Non functional requirements</td><td>• The system should protect account data securely.<br>• The interface should be simple and user-friendly.<br>• The system should respond quickly when updating accounts.</td></tr>
</table>

### 3.3.2 Configure System

- The system shall allow the admin to update system settings.
- The system shall allow the admin to configure fees, operating hours, and policies.
- The system shall save updated settings immediately.
- The system shall restrict configuration access to admin users only.
- The system shall display confirmation after successful changes.

| ***Use Case Name*** | | Configure System |
|---|---|---|
| Actors | | Admin |
| Preconditions | | Admin is logged into the system. |
| Normal Flow | Description | 1. The admin opens the system configuration page.<br>2. The system displays current settings.<br>3. The admin modifies required settings.<br>4. The admin saves the new configuration.<br>5. The system updates the settings and displays confirmation. |
| | Postconditions | System settings are successfully updated. |
| Alternative flows and exceptions | | If invalid values are entered, the system displays an error message. |
| Non functional requirements | | • The system should store settings securely.<br>• The interface should be simple and user-friendly.<br>• The system should apply changes efficiently. |

### 3.3.3 View Reports & Analytics

- The system shall allow the admin to view reports and analytics.
- The system shall display sales, user, and vendor performance data.
- The system shall allow filtering reports by date or category.
- The system shall allow exporting reports in PDF or CSV format.
- The system shall restrict report access to admin users only.

| ***Use Case Name*** | | View Reports & Analytics |
|---|---|---|
| Actors | | Admin |
| Preconditions | | Admin is logged into the system and report data exists. |
| Normal Flow | Description | 1. The admin opens the reports and analytics page.<br>2. The system retrieves available data.<br>3. The admin selects report type or date range.<br>4. The system displays charts, summaries, and statistics.<br>5. The admin reviews or exports the report. |
| | Postconditions | The admin successfully views reports or exports analytics data. |
| Alternative flows and exceptions | | If no data is available, the system displays a message. |
| Non functional requirements | | • The system should generate reports accurately.<br>• The interface should be simple and user-friendly.<br>• The system should load reports efficiently. |

## 3.4 Staff

### 3.4.1 Call Queue Number

- The system shall allow staff to call the next queue number.
- The system shall display the called queue number on the customer screen.
- The system shall allow staff to recall a queue number if needed.
- The system shall update the queue list after a number is called.
- The system shall restrict queue control functions to staff users only.

| ***Use Case Name*** | | Call Queue Number |
|---|---|---|
| Actors | | Staff |
| Preconditions | | The staff is logged into the system and queue numbers are available. |
| Normal Flow | Description | 1. The staff opens the queue management page.<br>2. The system displays the waiting queue list.<br>3. The staff selects the next queue number.<br>4. The system announces and displays the selected queue number.<br>5. The queue list is updated automatically. |
| | Postconditions | The selected queue number is successfully called and updated. |
| Alternative flows and exceptions | | If no queue numbers are available, the system displays a message. |
| Non functional requirements | | • The system should update the queue in real time.<br>• The interface should be simple and user-friendly.<br>• The system should respond quickly when calling numbers. |

### 3.4.2 Confirm Pickup / Delivery

- The system shall allow staff to confirm completed pickup orders.
- The system shall allow staff to confirm completed delivery orders.
- The system shall update order status after confirmation.
- The system shall notify customers when confirmation is completed.
- The system shall restrict confirmation functions to staff users only.

| ***Use Case Name*** | | Confirm Pickup / Delivery |
|---|---|---|
| Actors | | Staff |
| Preconditions | | The staff is logged into the system and active orders exist. |
| Normal Flow | Description | 1. The staff opens the order confirmation page.<br>2. The system displays active pickup and delivery orders.<br>3. The staff selects a completed order.<br>4. The staff confirms pickup or delivery.<br>5. The system updates the order status and records completion. |
| | Postconditions | The selected order is marked as completed successfully. |
| Alternative flows and exceptions | | If the selected order is invalid or already completed, the system displays an error message. |
| Non functional requirements | | • The system should update status accurately.<br>• The interface should be simple and user-friendly.<br>• The system should process confirmations quickly. |

# 4 System Models

## 4.1 Business Rules

(1:M) A Student may place many orders. (active)
Many orders may be placed by one Student. (passive)

(1:M) A Vendor may fulfill many orders. (active)
Many orders may be fulfilled by one Vendor. (passive)

(1:M) A Vendor may manage many menu items. (active)
Many menu items may belong to one Vendor. (passive)

(1:M) A Vendor may define many time slots. (active)
Many time slots may belong to one Vendor. (passive)

(1:M) An Order may contain many order items. (active)
Many order items may belong to one Order. (passive)

(1:M) A MenuItem may be included in many order items. (active)
Many order items may reference one MenuItem. (passive)

(1:1) An Order must have exactly one payment. (active)
One payment must belong to exactly one Order. (passive)

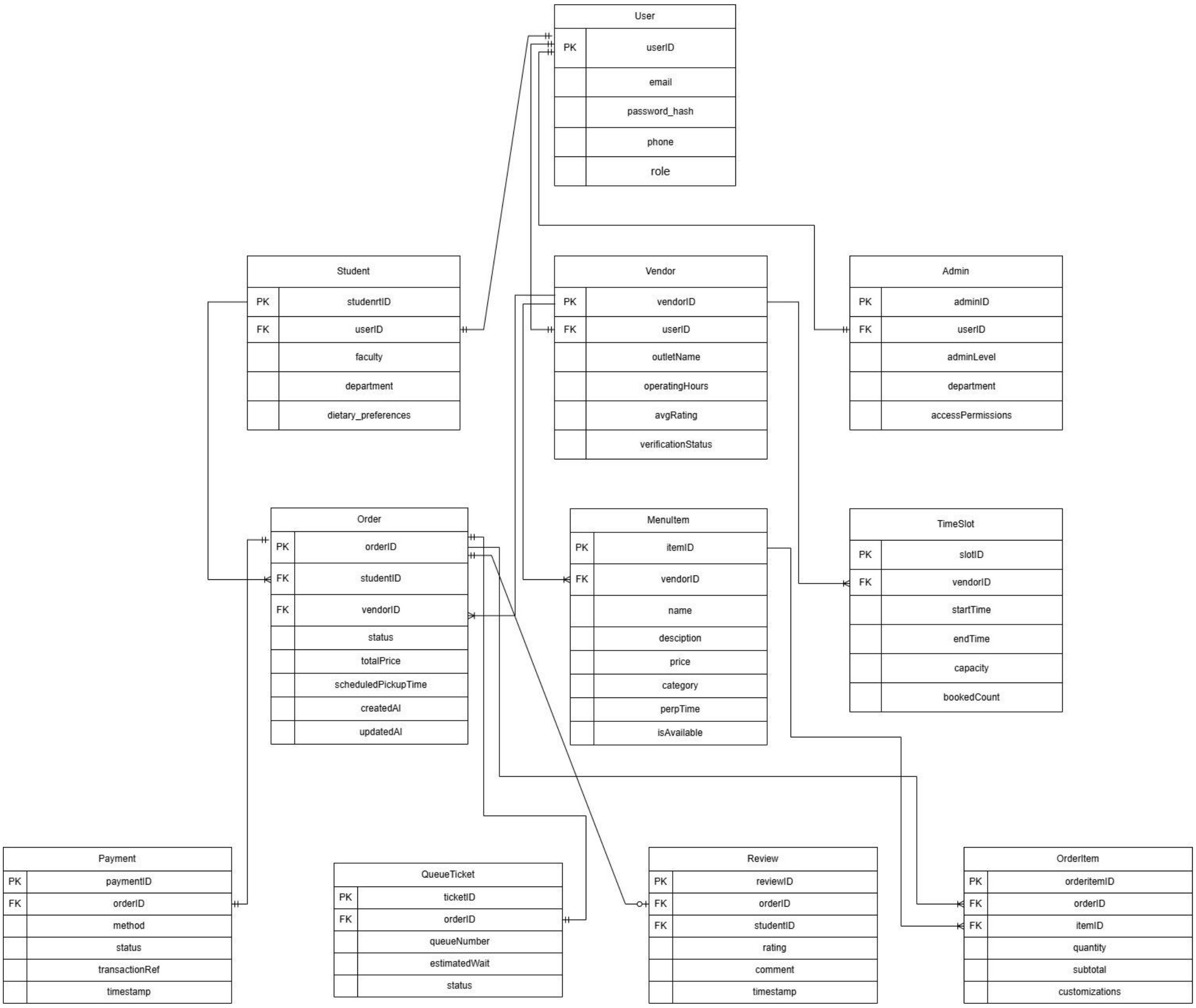
(1:1) An Order may be assigned exactly one queue ticket. (active)
One queue ticket must belong to exactly one Order. (passive)

(0:1) An Order may have at most one review. (active)
One review must belong to exactly one Order. (passive)

(1:1) A User account must have exactly one role. (active)
One role assignment must belong to exactly one User. (passive)

(1:M) A Student may submit many reviews. (active)
Many reviews may be submitted by one Student. (passive)

## 4.2 Class Diagrams / ERD



## 4.3 Classes / Entities

| Class / Entity | Description |
|---|---|
| User | Base authentication entity storing userID, email, password_hash, phone, and role (Student/Vendor/Admin/Staff). Serves as parent for role-specific entities. |
| Student | Inherits from User. Stores student-specific studentID, faculty, department, and dietary_preferences. Links to orders placed and reviews submitted. |
| Vendor | Inherits from User. Contains vendorID, outletName, operatingHours, avgRating, and verificationStatus. Manages menu items, time slots, and fulfills orders. |
| Admin | Inherits from User. Holds adminID, adminLevel, department, and accessPermissions. Responsible for system configuration and user/vendor management. |
| MenuItem | Represents food/drink offerings with itemID, name, description, price, category, prepTime, and isAvailable status. Owned by a specific Vendor. |
| Order | Core transaction entity storing orderID, status, totalPrice, scheduledPickupTime, and timestamps. Links Student, Vendor, and associated order items. |
| OrderItem | Junction entity connecting Order and MenuItem. Tracks orderItemID, quantity, subtotal, and customizations for each item in an order. |
| Payment | Records financial transactions with paymentID, method, status, transactionRef, and timestamp. One-to-one relationship with Order. |
| QueueTicket | Manages order fulfillment queue with ticketID, queueNumber, estimatedWait, and status (Waiting/Called/Served). Assigned to each Order. |
| Review | Captures post-order feedback containing reviewID, rating (1-5), comment, and timestamp. Submitted by Student for completed Orders. |
| TimeSlot | Defines vendor availability windows with slotID, startTime, endTime, capacity, and bookedCount. Enables scheduled ordering functionality. |

**Relationships:**

- User 1..1 Student (Student profile inheritance)
- User 1..1 Vendor (Vendor profile inheritance)
- User 1..1 Admin (Admin profile inheritance)
- Student 1..* Order (Student places orders)
- Vendor 1..* Order (Vendor fulfills orders)
- Vendor 1..* MenuItem (Vendor manages menu catalog)
- Vendor 1..* TimeSlot (Vendor defines scheduling windows)
- Order 1..* OrderItem (Order contains line items)
- MenuItem 1..* OrderItem (MenuItem referenced in orders)
- Order 1..1 Payment (Order settles transaction)
- Order 1..1 QueueTicket (Order assigned queue position)
- Order 0..1 Review (Order may receive feedback)

# 5 Non-Functional Requirements

- **Performance:** System shall support up to 500 concurrent users during peak hours with page load times under 2 seconds. Real-time order status updates shall reflect within 1 second.
- **Security:** All data in transit shall be encrypted via TLS 1.3. Passwords stored using bcrypt hashing. Role-Based Access Control (RBAC) enforced. Payment processing compliant with PCI-DSS standards.
- **Usability:** Intuitive, mobile-responsive interface adhering to WCAG 2.1 AA accessibility standards. First-time users shall complete registration and place an order within 3 minutes.
- **Reliability & Availability:** 99.5% uptime during campus operating hours (06:00–22:00). Automated daily database backups with point-in-time recovery capability.
- **Scalability:** Cloud-native architecture allowing horizontal scaling of application and database servers to handle seasonal enrollment increases or event-based demand spikes.

***Maintainability:*** *Modular codebase with comprehensive logging, API documentation, and version control. Clear separation of frontend, backend, and database layers.*

# 6 References


- IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications.
- Campus Food Service Operational Guidelines (Internal University Document, 2025).
- Payment Gateway Integration Documentation (Stripe/PCI Compliance Standards).
- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson.
- OWASP Foundation (2024). Password Storage Cheat Sheet. https://cheatsheetseries.owasp.org
- World Wide Web Consortium (2018). Web Content Accessibility Guidelines (WCAG) 2.1.
- diagrams.net (draw.io). Online Diagramming Tool for ERD and Use Case Diagrams.